Cricket Simulator

OS Project

Github Link | Demo Video

24114005 24114088
24114050 24114006
24114028

Considerations - To explain design choices

A. Resource and Entity Mapping

- The pitch as the critical section: The cricket pitch is modeled as the system's primary Critical Section. Because a ball cannot be bowled while a batsman is hitting, and a batsman cannot hit until a ball is bowled, strictly one action can occur on the pitch at a time. This is protected by the (pitch_mutex).
- Players as concurrent threads: Bowlers (Producers), Batsmen (Consumers), and Fielders (Waiters) are modeled as independent POSIX threads. They execute concurrently but must synchronize their access to the shared state (the pitch and the scoreboard).
- The pavilion as a counting semaphore: The crease can hold exactly two batsmen. To model the pavilion waiting area, we used a semaphore (crease_sem) initialized to 2. This naturally restricts access so that only two batsman threads can actively compete for the pitch at any given time, while the rest block in the queue.

B. Process Synchronization & State Management

- Strict Two-Way Handshake: To prevent race conditions during complex events (like wide or no ball), we applied a strict handshake protocol. The producer (bowler) utilizes (pthread_cond_wait) to sleep until the consumer (batsman) successfully processes the delivery and broadcasts a wakeup signal, ensuring exact ball-by-ball atomicity.
- Separation of Locks: Updating the scoreboard requires accessing shared global variables (Global_Score, bowler_runs, etc.). We applied a (score_mutex) separately from the (pitch_mutex) to prevent dirty reads and ensure thread-safe stat aggregation.

C. CPU Scheduling Strategies

- Bowler rotation (Round Robin): A pre-defined token array ensures Bowlers 1-6 strictly serve their 6-ball quantum/overs before a context switch is enforced.
- Batting order (SJF): The batting lineup arrival is managed by an SJF scheduling algorithm. Batsmen threads are sorted based on their predicted burst time (estimated balls faced), allowing tail-enders to enter the crease before the top order.

D. Deadlock Detection and Recovery

- The Run-Out (Circular Wait): A deadlock occurs during a running "mix-up" (7% probability on running plays). Batsman A currently holds End 1 and requests End 2, while Batsman B currently holds End 2 and requests End 1. Because neither thread will release its current resource until it acquires the target resource, the system deadlocks.
- The Third Umpire Monitor: The Third Umpire acts as the OS Kernel. It monitors the (Allocation) and (Request) matrices. When a cycle is detected, it executes Process Termination, forcefully selecting one batsman thread as the run-out victim to break the cycle and recover the system.

Assumptions - To make simulation realistic

A. Match Format

- T20 Constraints: The simulation strictly adheres to a max 20 overs (120 balls) and 10 wickets per innings. If 10 wickets fall before 20 overs are completed, the Kernel (Umpire) immediately terminates all active threads and ends the simulation.
- Bowler working (Round Robin): We assumed that there are 6 bowlers available. To satisfy the T20 rule of a 4 over max per bowler, the RR scheduler uses a deterministic token array where bowlers 1 and 2 are assigned exactly 4 overs, and bowlers 3, 4, 5, and 6 are assigned exactly 3 overs.

B. CPU Burst Times & SJF Execution

- Predicted vs Actual execution: To implement SJF scheduling, we assumed a predicted burst time (estimated balls faced) for every batsman. Top-order batsmen are assumed to have high burst times (like 50-60 balls), while tail-enders are assigned low burst times (like 2-10 balls).
- Non-Preemptive: As SJF scheduler is non preemptive, once a batsman thread acquires the CPU (enters the crease), it cannot be forcibly removed by the OS until it naturally terminates (gets out), even if its actual execution time far exceeds its predicted burst time.

C. Delivery Probabilities (System Interrupts & Outputs)

- Stratified Probability Distribution**:** To mathematically enforce the SJF burst time predictions, batsman threads are divided into three tiers: Top Order (1-3), Middle-Order (4-7), and Tail-Enders (8-11). Each tier calls a specific `simulate_delivery()` function with distinct probabilities. For example, Tail-enders have a 20% combined chance of getting out per ball, while Top-Order batsmen only have a 6% chance. It is implemented as shown in below images.

- Extras (Wides & No-Balls): Wide (3%) and no balls (3%) are treated as system interrupts. An extra run is added to the total, but the ball is deemed "illegal" and does not decrease the bowler's time quantum (the 6-ball over limit).

```
int simulate_delivery1to3() {   // batsmen 1 to 3
    int r = rand() % 100;
    if (r < 3) return -3;       // 3%  wide
    if (r < 6) return -4;       // 3%  no Ball
    if (r < 31) return 0;       // 25% dot ball
    if (r < 66) return 1;       // 35% 1 run
    if (r < 81) return 2;       // 15% 2 runs
    if (r < 84) return 3;       // 3%  3 runs
    if (r < 91) return 4;       // 7%  4 runs
    if (r < 94) return 6;       // 3%  6 runs
    if (r < 97) return -1;      // 3%  bowled
    return -2;                  // 3%  catch out

    // below we added additional 7% prob of run out if 1,2,3 runs taken
    // so run out prob = 7% of 53% = 3.71% approx
}
```

```
int simulate_delivery4to7() {   // batsmen 4 to 7
    int r = rand() % 100;
    if (r < 3) return -3;       // 3%  wide
    if (r < 6) return -4;       // 3%  no Ball
    if (r < 31) return 0;       // 25% dot ball
    if (r < 60) return 1;       // 29% 1 run
    if (r < 75) return 2;       // 15% 2 runs
    if (r < 78) return 3;       // 3%  3 runs
    if (r < 85) return 4;       // 7%  4 runs
    if (r < 88) return 6;       // 3%  6 runs
    if (r < 94) return -1;      // 6%  bowled
    return -2;                  // 6%  catch out
}
```

```
int simulate_delivery8to11() {  // batsmen 8 to 11
    int r = rand() % 100;
    if (r < 3) return -3;       // 3%  wide
    if (r < 6) return -4;       // 3%  no Ball
    if (r < 31) return 0;       // 25% dot ball
    if (r < 58) return 1;       // 27% 1 run
    if (r < 68) return 2;       // 10% 2 runs
    if (r < 71) return 3;       // 3%  3 runs
    if (r < 77) return 4;       // 6%  4 runs
    if (r < 80) return 6;       // 3%  6 runs
    if (r < 90) return -1;      // 10% bowled
    return -2;                  // 10% catch out
}
```

D. Deadlock Induction (Run-Out)

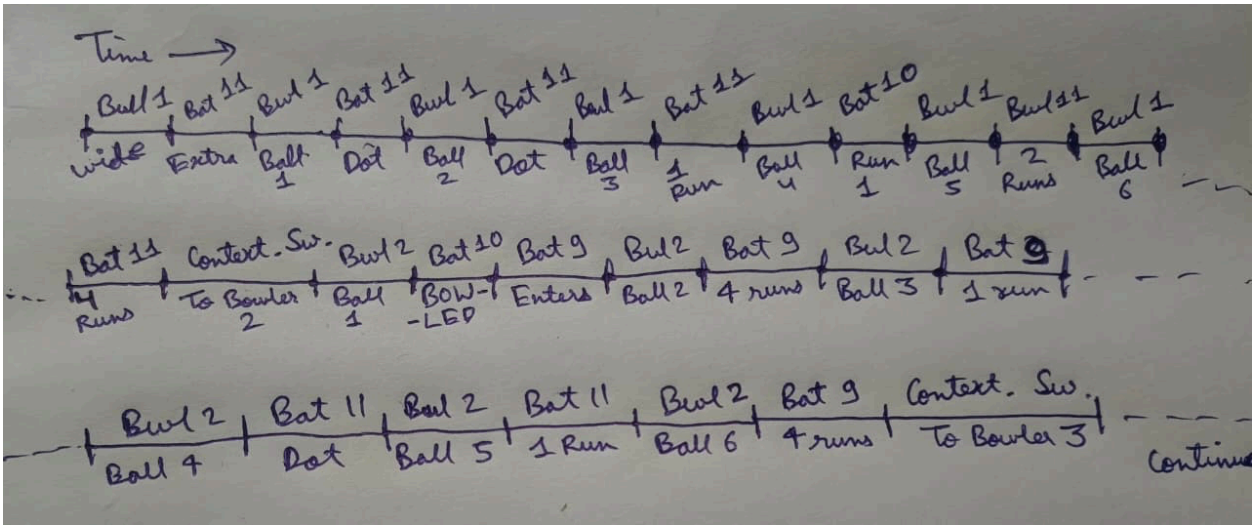
- Mix-Up Probability: A circular wait deadlock does not happen on boundaries or dot balls. It only occurs on running plays (1, 2, or 3 runs) with a probability we assumed to be 7%.
- Random Process Termination: When the third umpire detects this deadlock, it is assumed that the system recovers by randomly selecting one of the two deadlocked batsman threads (50/50 chance for the striker or non-striker) to be terminated (run out).

Visualization - Gantt chart

The gantt chart maps the thread execution sequence holding the Pitch Mutex. In this OS mapping, the Pitch is the CPU, and the processes alternate context strictly between the bowler and the striker.
We have took 2 overs from a sample match from our simulation, its starting logs is like this :-

[Batsman 11] entered the crease. Wait Time: 0.02 ms
[Batsman 10] entered the crease. Wait Time: 2.59 ms
[Bowler 1] delivering Over 1, Ball 1...
--- WIDE! Extra run added. Ball must be re-bowled. (Total: 1/0)
[Bowler 1] delivering Over 1, Ball 1...
[Batsman 11] played a DOT ball. (Total: 1/0)
[Bowler 1] delivering Over 1, Ball 2...
[Batsman 11] played a DOT ball. (Total: 1/0)
[Bowler 1] delivering Over 1, Ball 3...
[Batsman 11] hit for 1 runs! (Total: 2/0)
[Bowler 1] delivering Over 1, Ball 4...
[Batsman 10] hit for 1 runs! (Total: 3/0)
[Bowler 1] delivering Over 1, Ball 5...
[Batsman 11] hit for 2 runs! (Total: 5/0)
[Bowler 1] delivering Over 1, Ball 6...
[Batsman 11] hit for 4 runs! (Total: 9/0)
[Umpire] Over 1 complete! Strike rotated.
[Umpire] Context switch! Bowler 2 loading...
[Bowler 2] delivering Over 2, Ball 1...
!! [Batsman 10] is BOWLED! !!
[Batsman 10] walks back to the pavilion.
[Batsman 9] entered the crease. Wait Time: 355.43 ms
[Bowler 2] delivering Over 2, Ball 2...
[Batsman 9] hit for 4 runs! (Total: 13/1)
[Bowler 2] delivering Over 2, Ball 3...
[Batsman 9] hit for 1 runs! (Total: 14/1)
[Bowler 2] delivering Over 2, Ball 4...
[Batsman 11] played a DOT ball. (Total: 14/1)
[Bowler 2] delivering Over 2, Ball 5...
[Batsman 11] hit for 1 runs! (Total: 15/1)
[Bowler 2] delivering Over 2, Ball 6...
[Batsman 9] hit for 4 runs! (Total: 19/1)
[Umpire] Over 2 complete! Strike rotated.
[Umpire] Context switch! Bowler 3 loading...

This is the gantt chart for these two overs :-



Analysis - Batting order strategies

A. The Scheduling Prediction (SJF) - As an OS can't see the future, it must predict burst times to create a sorted Ready queue. To model this, the simulation assigns predefined burst estimates based on the batting order (e.g., Top Order = 50 balls, Tail enders = 5 balls). The scheduler uses these predictions to successfully sort the threads, granting the tail-enders priority access to the (crease_sem) semaphore.

B. The Execution Reality (Uniform Processing) Once a Batsman thread acquires the pitch (CPU), the simulation abandons the static predicted burst time and subjects the thread to its respective tiered probability distribution. This design choice was made for two critical reasons:

1. **Statistical Accuracy:** By grouping threads into three probability tiers, the code guarantees that tail-enders will, on average, genuinely have much shorter CPU bursts than top-order batsmen. This ensures our SJF vs. FCFS wait time analysis is based on empirical execution differences, not just sorting order.
2. **Simulating Real-World OS Variance:** Although probabilities are weighted, the actual execution remains random. A top-order batsman might still get out on the first ball (a surprisingly short CPU burst), or a tail-ender might survive for 20 balls (exceeding its prediction). Because our SJF implementation is non-preemptive, the thread retains control of the CPU until a natural termination event occurs, perfectly mirroring how real non-preemptive processes behave when they outlive their initial OS estimates.

C. Wait Time Analysis (SJF vs FCFS)
To implement SJF, run the function `apply_sjf_scheduling()` in our code, and to implement FCFS, just comment it out.
For middle order batsmen (6 and 7), we took 3 matches with SJF and 3 with FCFS, here's the waiting time data :-

Simulation	FCFS - Bats 6	FCFS - Bats 7	SJF - Bats 6	SJF - Bats 7
Match 1	1323.58 ms	2520.33 ms	1053.02 ms	671.01 ms
Match 2	1054.18 ms	2028.67 ms	1430.99 ms	942.43 ms
Match 3	1859.61 ms	2343.02 ms	621.81 ms	568.30 ms
Avg	**1412.46 ms**	**2297.34 ms**	**1035.27 ms**	**727.35 ms**

Conclusion: The data clearly demonstrates the Convoy Effect present in FCFS. Under FCFS, the middle order threads are forced to wait in the pavilion while the Top-Order threads monopolize the (crease_sem) semaphore, resulting in high average wait times.
Conversely, the SJF scheduler prioritizes the Tail-ender threads. Because our stratified probability functions guarantee that Tail-enders have shorter CPU bursts, they acquire and release the semaphore very quickly. Consequently, the middle order threads move up the queue and enter the crease significantly faster, drastically reducing their average wait times.